Hochschule Kaiserslautern
University of Applied Sciences

Informatik und Mikrosystemtechnik
Zweibrücken

Studiengang

Angewandte Informatik

PO Version 2018

Bachelorarbeit

Entwicklung und Implementierung einer Web-Anwendung zur Konvertierung von Dokumenten in maschinenlesbare Formate mittels Künstlicher Intelligenz

Development and Implementation of a Web Application for Document Conversion into Machine-Readable Formats Using Artificial Intelligence

vorgelegt von

Daniel Hohmann

06. März 2026

Betreuung: Dr.-Ing. Hubert Zitt

Zweitkorrektor: Prof. Dr. Manh Tien Tran

Ehrenwörtliche Erklärung

Hiermit erkläre ich, Daniel Hohmann, geboren am 20.01.1998 in Mannheim, ehrenwörtlich, dass ich meine Bachelorarbeit mit dem Titel:

Entwicklung und Implementierung einer Web-Anwendung zur Konvertierung von Dokumenten in maschinenlesbare Formate

selbstständig und ohne fremde Hilfe angefertigt und keine anderen als in der Abhandlung angegebenen Hilfen benutzt habe.

Die Übernahme wörtlicher Zitate aus der Literatur sowie die Verwendung der Gedanken anderer Autoren an den entsprechenden Stellen habe ich innerhalb der Arbeit gekennzeichnet.

Falls bei der Erstellung dieser Arbeit Werkzeuge mit Künstlicher Intelligenz (KI) genutzt wurden, ist deren Einsatz im Anhang vollständig aufgeführt. Sofern von KI-Werk-zeugen generierte Inhalte in die Arbeit übernommen wurden, sind diese entsprechend gekennzeichnet.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben kann.

_____ _____

Ort den Unterschrift

Kurzfassungen

Kurzfassung in Deutsch

Die Deutsche Rentenversicherung (DRV) sieht sich aufgrund des demografischen Wandels mit einer steigenden Anzahl eingehender Leistungsanträge bei gleichzeitig sinkender Verfügbarkeit von Fachkräften konfrontiert. Vor diesem Hintergrund wird in dieser Bachelorarbeit die Entwicklung und Implementierung einer Web-Anwendung zur Konvertierung von Dokumenten in maschinenlesbare Formate mittels Künstlicher Intelligenz untersucht.

Ziel der Arbeit ist es, einen Beitrag zur intelligenten Dokumentenverarbeitung im Rahmen des Modernisierungsprojekts "Modernisierungsprojekt des digitalen Antragsverfahrens der DRV (eAQ:modern)" zu leisten. Die entwickelte Anwendung ermöglicht das Hochladen gescannter Dokumente im PDF-Format, deren automatisierte Verarbeitung mittels Optical Character Recognition (OCR) sowie die Transformation in strukturierte Datenformate wie JavaScript Object Notation (JSON) oder Markdown. Die extrahierten Informationen können überprüft, korrigiert und für eine weitere automatisierte Verarbeitung genutzt werden, beispielsweise zur Befüllung digitaler Antragsformulare.

Die Web-Anwendung wurde als Single-Page Application (SPA) umgesetzt. Das Frontend basiert auf Angular und TypeScript unter Verwendung des DRV-Designsystems, während das Backend mit dem Java-Framework Spring realisiert wurde. Für die Dokumentenanalyse kommt die Open-Source-Software Docling zum Einsatz; zusätzlich kann ein Large Language Model (LLM) über eine Application Programming Interface (API) angebunden werden.

Die entwickelte Lösung zeigt, wie Künstliche Intelligenz (KI)-gestützte Verfahren zur Effizienzsteigerung beitragen und Mitarbeitende bei der zeitintensiven manuellen Dokumentenverarbeitung nachhaltig entlasten können.

Kurzfassung in Englisch

Due to demographic change, the German Pension Insurance (Deutsche Rentenversicherung (DRV)) faces an increasing number of benefit applications while simultaneously experiencing a shortage of skilled personnel. This bachelor thesis addresses this challenge by developing and implementing a web application for converting documents into machine-readable formats using artificial intelligence.

The objective of this work is to contribute to intelligent document processing within the modernization project “eAQ:modern”. The developed application enables users to upload scanned Portable Document Format (PDF) documents, which are processed using Optical Character Recognition (OCR) and transformed into structured data formats such as JSON or Markdown. The extracted information can be reviewed, corrected, and reused for automated processing, for example to populate digital application forms.

The application was implemented as a single-page application. The frontend was developed using Angular and TypeScript in accordance with the DRV design system, while the backend was built with the Java framework Spring. Document analysis is performed using the open-source tool Docling, and a Large Language Model (LLM) can be integrated via API.

The results demonstrate how AI-supported document processing can significantly increase efficiency and reduce the workload associated with manual document handling in public administration.

Inhaltsverzeichnis

1 Einleitung **3**
1.1 Motivation . . . 3
1.2 Problemstellung und Zielsetzung . . . 4

2 Theorie **7**
2.1 Webanwendungen . . . 7
2.1.1 Single-Page Application (SPA) . . . 7
2.1.2 Architekturmuster . . . 8
2.2 Intelligente Dokumentenverarbeitung . . . 10
2.2.1 Optical Character Recognition (OCR) . . . 10
2.2.2 Retrieval Augmented Generation (RAG) . . . 12
2.2.3 Chunking . . . 12
2.2.4 Tokenization . . . 13
2.2.5 Embedding . . . 13
2.2.6 Layoutanalyse und semantische Strukturierung von Dokumenten 14
2.2.7 Human-in-the-Loop . . . 15
2.3 Modernisierung der Datenstelle der Deutschen Rentenversicherung (DSRV) 16
2.3.1 Neuentwicklung der digitalen Antragstellung . . . 19

3 Projektdurchführung **23**
3.1 Projektbeschreibung . . . 23
3.2 Technische Rahmenbedingungen . . . 24
3.3 Anwendung - Frontend mit Angular . . . 25
3.3.1 Aufbau der SPA . . . 26
3.4 Softwarearchitektur . . . 28
3.5 Designsystem . . . 28
3.6 Versionierung . . . 28
3.7 Anwendung - Backend mit Spring-Boot . . . 28
3.8 Anwendung - Containerisierung . . . 28

4 Zusammenfassung **29**

5 Ausblick **31**

Abbildungsverzeichnis **33**

Literaturverzeichnis **33**

Abkürzungsverzeichnis **35**

1 Einleitung

1.1 Motivation

Seit fast 150 Jahren existiert in Deutschland eine gesetzliche Rentenversicherung. Im Jahr 2005 wurde die DRV gegründet, in der alle gesetzlichen Rentenversicherungen vereinigt sind. Im Laufe dieser Zeit hat sich eine große Menge expliziten Wissens angesammelt, das zu einem erheblichen Teil nicht maschinell verarbeitet werden kann, da es nicht in geeigneter strukturierter Form vorliegt. Darüber hinaus werden täglich Informationen gesammelt, auf die in der Zukunft wieder zugegriffen werden muss. Die Transformation dieses Wissens, das teilweise in Form von physischen Papieren und teilweise in Form von unstrukturierten Dokumenten vorliegt, ist aus verschiedenen Gründen von besonderem Interesse. Zum einen erfolgt die Suche nach Informationen beispielsweise für Mitarbeiter wesentlich schneller, wenn die Wissensbasis in geeigneter Form vorliegt und sich maschinell durchsuchen lässt. Zum anderen ist die automatisierte Umwandlung von Dokumenten durch Maschinen deutlich schneller und weniger fehleranfällig als die manuelle Umwandlung durch den Menschen. Außerdem kann das Wissen besser integriert werden in einen digitalisierten Prozess der schon in großen Teilen Daten automatisiert verarbeitet.

Das Ziel dieser Bachelorarbeit ist die Entwicklung einer Web-Anwendung, die die Mitarbeiter bei der Transformationen von Informationen unterstützt. Die Anwendung besitzt eine übersichtliche Benutzeroberfläche, die responsiv gestaltet ist und sich auf dem Laptop und dem Smartphone bedienen lässt. Eine geführte Bedienung durch die einzelnen Schritte vereinfacht die Bedienung und leitet den Nutzer. Der Nutzer lädt zuvor gescannte Dokumentenstapel hoch. Anschließend werden die Dokumente analysiert und verarbeitet. In einem weiteren Schritt kann der Nutzer die erkannten Informationen prüfen und gegebenenfalls korrigieren. Im letzten Schritt werden die Dokumente abgespeichert und bereitgehalten für eine erneute Nutzung.

Die Anwendung unterstützt die Mitarbeiter in den Beratungsstellen in ihrer täglichen Arbeit. Der Prozess der Dokumentenverarbeitung wird beschleunigt und erlaubt es dem Mitarbeiter, sich intensiver um die persönlichen Belange des Versicherten zu kümmern. Dadurch wird der Mitarbeiter entlastet und der Versicherte erhält einen besseren Service.

1.2 Problemstellung und Zielsetzung

Die Deutsche Rentenversicherung (DRV) bearbeitet pro Jahr rund 1,8 Millionen Anträge für verschiedenste Leistungen aber hauptsächlich für die Regelaltersrente (Deutsche Rentenversicherung, 2025, S. 67).
Wurden früher Rentenanträge noch in Papierform versendet und abgelegt, passiert das heute überwiegend digital. Die Erstellung von neuen Anträgen und die Aktualisierung von Anträgen nach gesetzlichen Vorgaben, sowie die Bearbeitung eingegangener Anträge erfordert eine enorme Arbeitsleistung durch die Mitarbeiter der Deutschen Rentenversicherung. Die Anzahl der eingehenden Anträge wird in den nächsten Jahren und Jahrzehnten aller Voraussicht nach weiter steigen. Dies zeigen nicht zuletzt die Erhebungen des statistischen Bundesamtes über die demographische Entwicklung in Deutschland (Lummer et al., 2025).
Die deutsche Rentenversicherung ist dadurch konfrontiert mit zwei gravierenden Problemen. Auf der einen Seite steigt die Anzahl der eingehenden Anträge durch eine steigende Zahl Regelaltersrente beantragender Personen. Andererseits verlassen viele Fachkräfte die DRV, weil sie die Grenze für die Regelaltersrente erreicht haben. Für die DRV bedeutet das gesteigerten Arbeitsaufwand bei geringerer Verfügbarkeit von Fachkräften. Nichtsdestotrotz müssen Anträge weiterhin in ausreichender Geschwindigkeit bearbeitet werden, sodass die Versicherten die ihnen zustehenden Leistungen beziehen können.

Der Boom im Bereich der generativen Künstliche Intelligenz (KI), der entstanden ist durch die Entwicklung von Chatbots, wie beispielsweise ChatGPT von OpenAI, hat bei den Entscheidungsträgern in der DRV Hoffnungen geweckt. Die KI soll das vorhandene Personal in ihrer Arbeit unterstützen, um Prozesse zu beschleunigen und den Abgang von Fachkräften kompensieren.

In dieser Arbeit wird die Arbeit von Hohmann (2025) aufgegriffen, welche eine Untersuchung zu den Einsatzmöglichkeiten von KI im Prozess der digitalen Antragstellung in der DRV zum Thema hat. Darin wird empfohlen, zunächst die KI-gestützte Indizierung von Anlagen umzusetzen, da sie das größte Unterstützungspotenzial bietet.

Beide Arbeiten werden für das Team eAQ:modern in der DRV verfasst, welches die Neuentwicklung des digitalen Antragsverfahrens“e-Antrag” zur Aufgabe hat. Durch die gesteigerte Anzahl an eingehenden Anträgen ist die Neuentwicklung des e-Antrags notwendig geworden, da die dort sich im Einsatz befindlichen, veralteten Technologien die große Anzahl an Anträgen nicht mehr bewältigen können.

In das Verfahren wird eine Anwendung integriert werden, welche die Antragsteller und die Mitarbeiter in den Beratungsstellen der DRV bei der Bearbeitung von Anträgen unterstützt. Die Antragsteller erhalten die Möglichkeit, Dokumente in Form von PDF-Dateien hochzuladen, welche anschließend durch die Anwendung in strukturierte Daten überführt werden. Die strukturierten Daten werden anschließend ausgewertet, um sie automatisiert durch die Anwendung in die entsprechenden Formularfelder des Antrags eintragen zu lassen. Dadurch wird die Antragstellung erheblich beschleunigt. Darüber hinaus wird die Anwendung Mitarbeiter bei der Einordnung von Dokumenten

unterstützen, die dem Antrag als Beleg für in Formulare eingegebene Information angehängt werden. Bei diesen Anhängen kann es sich um Dokumentenstapel handeln die völlig unsortiert und willkürlich gescannt sind. Die Lösung entlastet die Mitarbeiter bei der sehr zeitaufwendigen Sichtung dieser Dokumentenstapel deutlich.

Diese Bachelorarbeit gliedert sich in zwei Teile. Beschrieben wird zunächst die Funktionsweise einer Web-Anwendung und der intelligenten Dokumentenverarbeitung. Im ersten Teil werden zunächst die unterschiedlichen Teile beschrieben die eine Web-Anwendung benötigt und anschließend die Technologien hinter der intelligenten Dokumentenverarbeitung. Im zweiten Teil wird die Durchführung des Projekts beschrieben, in welchem eine Web-Anwendung entwickelt wird, mit Hilfe derer der Nutzer Daten aus gescannten physischen Papieren extrahieren kann. Die Anwendung bietet die Möglichkeit Dokumente so aufzubereiten, dass sie maschinell verarbeitet werden können und unterstützt Mitarbeiter bei der aufwändigen manuellen Dokumentenverarbeitung. Am Ende erfolgt eine Zusammenfassung der Arbeit und ein Ausblick.

Für die Entwicklung der Anwendung werden ähnliche Technologien verwendet wie für die eAntrag-Anwendung, um eine spätere Integration zu vereinfachen und es gelten die gleichen Anforderungen. Aus diesem Grund wird eine Web-Anwendung entwickelt, die im Browser verwendet werden kann. Dadurch ist keine spezielle Installation auf dem Gerät des Endnutzers notwendig. Das Frontend wird mit Hilfe des Frameworks Angular und der Programmiersprache TypeScript entwickelt. Die Benutzeroberfläche ist responsiv und lässt sich sowohl mit dem Computer als auch mit dem Smartphone bedienen. Für die Oberflächengestaltung wird das Designsystem der DRV verwendet. Das Backend wird mit Spring als modernes Java-Framework entwickelt. Die Verarbeitung von Dokumenten erfolgt durch die Open-Source Anwendung Docling. Zur Analyse der Dokumente kann ein beliebiges LLM per API-Key angebunden werden.

2 Theorie

2.1 Webanwendungen

Üblicherweise wird eine Webanwendung nach dem Client-Server-Modell betrieben. Das heißt, die Inhalte liegen auf einem Server und werden bei Bedarf an den Client gesendet. Die Kommunikation zwischen beiden Seiten erfolgt per Hypertext Transfer Protocol (HTTP). Den für den Nutzer sichtbaren Teil bezeichnet man als Frontend, den nicht-sichtbaren Teil als Backend. Das Frontend enthält alle Teile, die der Nutzer betrachten kann, die sogenannte Webseite. Der Browser fordert diese vom Server an und zeigt sie an. Im Backend wird die Datenhaltung organisiert und die Geschäftslogik implementiert. Eine Authentifizierungsmöglichkeit, Persistenz und API-Bereitstellung sind in den meisten Fällen auch notwendig.

Das Frontend besteht hauptsächlich aus Hypertext Markup Language (HTML)-Dateien, Cascading Style Sheets (CSS)-Dateien und JavaScript-Dateien. Die HTML-Datei enthält die Webseiten-Inhalte, die durch Tags versehen in eine hierarchische Struktur gegliedert wird. Die Tags nutzt der Browser um die Inhalte richtig anzuzeigen.

In einer CSS-Datei sind Designparameter für die entsprechenden Webseiten-Inhalte definiert. Mit Hilfe von Selektoren lässt sich das Aussehen von Inhalten in den dazugehörigen Tags anpassen. Dazu zählen unter anderem die Schriftart, -größe, und -stärke, aber auch beispielsweise die Farbe und Größe von Elementen.

JavaScript ist eine Skriptsprache, die der Webseite Dynamik verleiht. Mit JavaScript können komplexe Funktionen implementiert werden, um Daten auf der Webseite live zu verändern oder auszutauschen mit dem Backend. Dadurch kann der Nutzer mit der Webseite interagieren.

2.1.1 Single-Page Application (SPA)

Heutzutage werden häufig SPA eingesetzt, die nur aus einer einzelnen Seiten besteht. Bei Bedarf wird der Inhalt der Seite dynamisch verändert. Ein vollständiges Neuladen der Seite erfolgt nicht. Besonders vorteilhaft ist hierbei, dass sich die Seite für den Nutzer sehr schnell und ähnlich wie eine App anfühlt. Eine solche Seite ist besonders geeignet für das Smartphone, lässt sich aber auch auf dem Computer gut bedienen. In Abbildung 2.1 wird die Architektur einer SPA gezeigt.

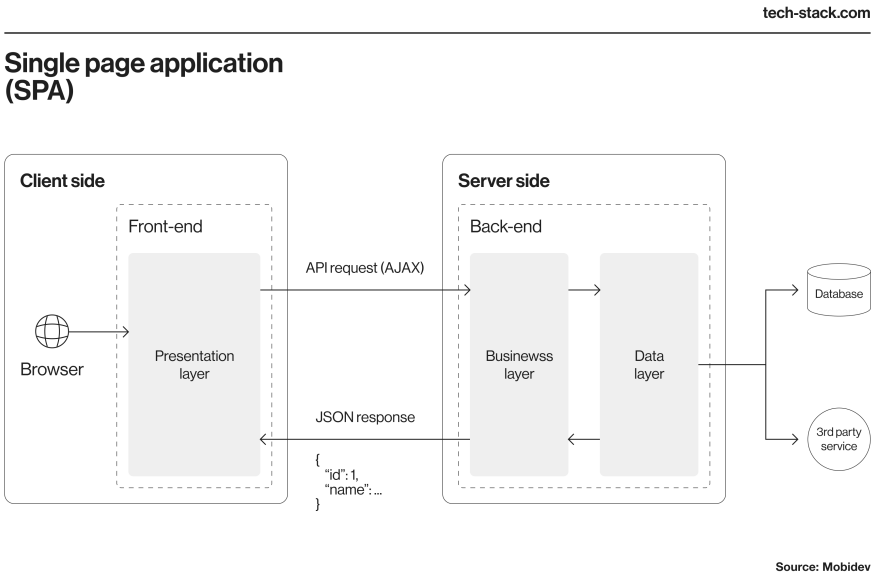

Abbildung 2.1: Architektur von Single-Page-Anwendungen (Tech-Stack.com, 2026)

Der clientseitige Browser fragt zunächst die Webseite vom Server an. Zurück erhält er eine HTML-Seite als eine Art Hülle, sowie CSS-Dateien und JavaScript-Code. Die HTML-Seite besteht oft nur aus einem initialen `<div id=‘‘app’’>`, in das dynamisch die restlichen Inhalte der Seite generiert werden. Zunächst parst der Browser das HTML und erzeugt das Document Object Model (DOM). Ein clientseitiger Router analysiert die URL und entscheidet, welche Komponente angezeigt werden soll. Bei Änderungen der URL erkennt dies der Router und veranlasst das entsprechende Laden einer Komponente. Gegebenenfalls wird eine HTTP-Anfrage an den Server gesendet, um Daten anzufragen. Sind alle Daten vorhanden, wird das DOM aktualisiert. Ein Großteil der Funktionalität erfolgt im Browser (Scott, 2015).

2.1.2 Architekturmuster

Ein etabliertes Prinzip in der Entwicklung ist die Aufteilung des Codes in verschiedene Funktionsbereiche. Dafür steht die Abkürzung “MV*”. M steht für Model, V für View und “*” als Platzhalter für einen austauschbaren dritten Part. Die Aufteilung des Codes erleichtert die Gestaltung, die Entwicklung und die Wartung der Codebasis. Verschiedene Ausprägungen dieses Konzepts sind Model-View-Controller (MVC), Model-View-Presenter (MVP) und Model-View-ViewModel (MVVM). Die meisten Implementierungen enthalten ein Model und eine View. Das Model enthält Daten, Geschäftslogik und Validierungslogik. Das Model repräsentiert die fachlichen Daten und unterstützt den Austausch von Daten mit der Geschäftslogik im Backend. Die View ist zuständig für die Anzeige der Daten in der Benutzeroberfläche. Sie zeigt den aktuellen Zustand des Models an und stellt die Benutzeroberfläche bereit.

Model-View-Controller (MVC)

Das MVC-Muster ist ein weit verbreitetes Architekturmuster. Neben der View und dem Model existieren Controller. Ein Controller dient häufig als Einstiegspunkt in die Anwendung und vermittelt zwischen dem Model und der View wie in der Abbildung 2.2 gezeigt wird. Der Controller verarbeitet Nutzereingaben und veranlasst daraufhin Änderungen im Model. Die View registriert die Veränderungen im Model und passt die Anzeige entsprechend der Veränderung an.

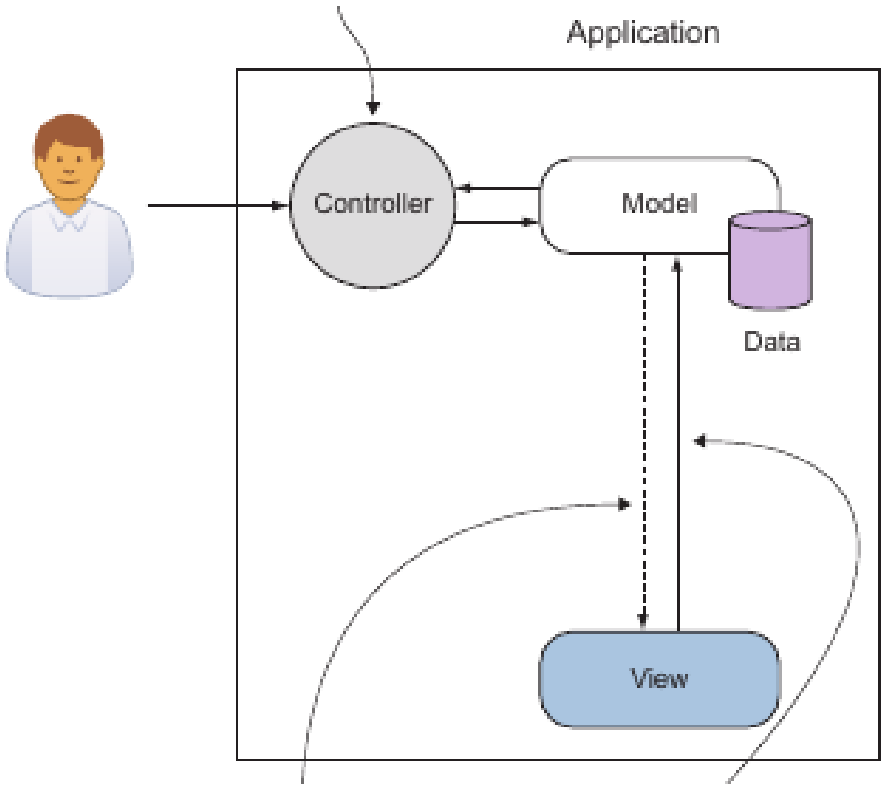

Abbildung 2.2: Das MVC-Architekturmuster (Scott, 2015, S. 25)

Model-View-Presenter (MVP)

Im MVP-Muster wird das Model stärker abgegrenzt von den anderen beiden Funktionsbereichen. Das Model enthält die fachlichen Daten. Jede View besitzt im Hintergrund einen Presenter. Im Presenter ist die Präsentationslogik enthalten. Die View reagiert auf Nutzereingaben, indem es die Aktionen an den Presenter übergibt, enthält aber selbst nur minimale Logik. Der Presenter verändert das Model und gibt Änderungen an die View weiter. Er agiert als "Vermittler" zwischen Model und View. Im Gegensatz zum MVC-Muster "beobachtet" die View nicht das Model. Aktualisierungen der View werden explizit durch den Presenter gesteuert.

Model-View-ViewModel (MVVM)

Bei diesem Architekturmuster dient die View als Einstieg in die Anwendung für den Nutzer. Die Komponente, die zwischen der View und dem Model vermittelt, ist das View-Model. Das View-Model ruft Daten aus dem Model ab, transformiert sie in ein darstellungsfähiges Format und stellt sie der View als Properties zur Verfügung. In Abbildung 2.3 wird gezeigt wie das View-Model View und Model verwaltet und synchronisiert.

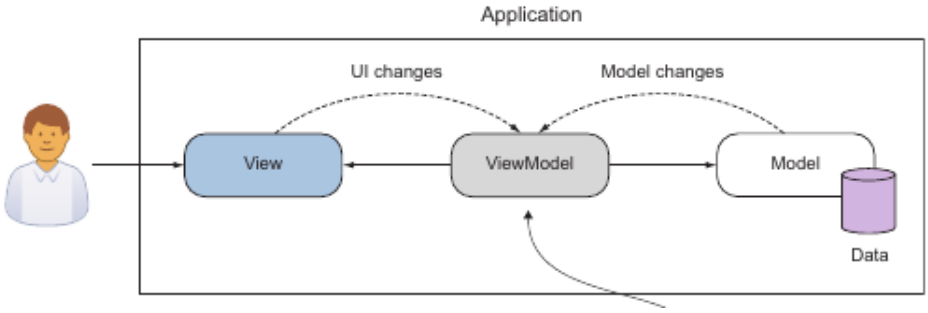

Abbildung 2.3: Das MVVM-Architekturmuster Scott, 2015, S. 27

2.2 Intelligente Dokumentenverarbeitung

Die Konvertierung von inhaltlich komplexen Dokumenten in maschinenlesbare ist seit vielen Jahren ein schwer lösbares Problem. Durch die Erfindung der Transformer-Architektur im Zusammenhang mit Large-Language-Models und damit zusammengehörigen Technologien wie Retrieval-Augmented-Generation sind die Lösungen in diesem Bereich besser geworden. Die Herausforderung besteht darin unterschiedliche Formate, komplexe Darstellungen von Informationen in Bildern und Grafiken zu analysieren und zu transformieren (Auer et al., 2024). Betroffen von diesem Problem ist eine Mehrzahl von Unternehmen, denn in jedem Unternehmen sind Dokumente in physischer Form im Umlauf. Jede Firma muss Briefe unter einer eingetragenen Adresse empfangen können, um für Behörden, Gerichte oder Geschäftspartner erreichbar zu sein. Aus dieser Tatsache lässt sich schlussfolgern, dass jede Firma physische Dokumente in irgendeiner Form verwalten muss. Da die manuelle Verarbeitung von Dokumenten zeitintensiv und fehleranfällig ist, werden diese immer häufiger automatisiert durch Maschinen verwaltet.

2.2.1 Optical Character Recognition (OCR)

Etablierte Ansätze zur automatisierten Verarbeitung von Dokumenten stützen sich auf einen mehrstufigen Prozess zur Umwandlung von physischen Dokumenten in maschinenlesbaren Text, der in Abbildung 2.4 skizziert ist.

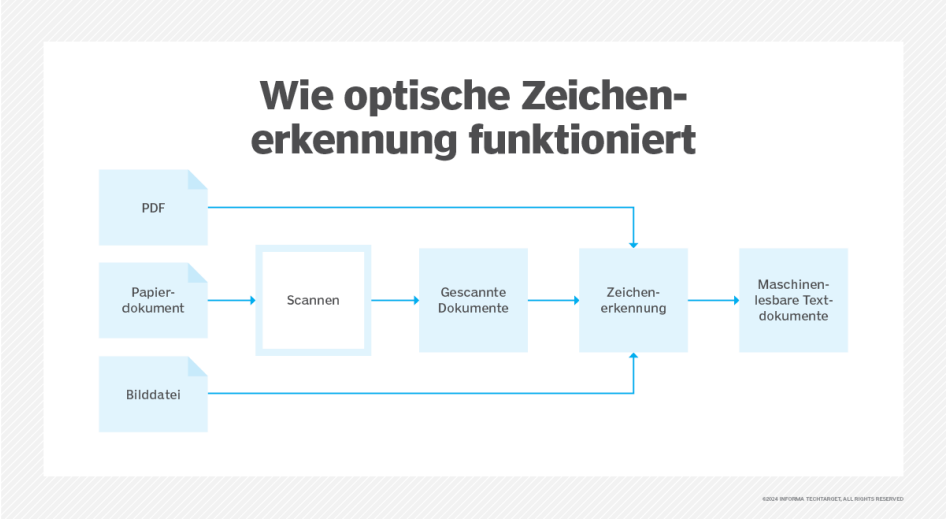

Abbildung 2.4: Transformation von physischen Dokumenten (ComputerWeekly.de, 2025)

Zunächst werden die physischen Dokumente digitalisiert. Hierzu werden sie eingescannt und als digitale PDF-Datei gespeichert. Häufig erfolgt eine Verbesserung der Qualität, indem der Kontrast erhöht wird oder Rauschen entfernt wird (ComputerWeekly.de, 2025). ISO-standardized Portable Document Format for Archiving (PDF/A) (PDFA) ist seit Jahren ein verbreiteter International Organization for Standardization (ISO)-normierter Standard zur Archivierung, der langfristige Lesbarkeit gewährleistet. Das Dokument im PDF-Format ist unterteilt in mehrere Objekte. Zusätzlich wird deren Anordnung und Struktur erfasst. Die Objekte werden als 8-bit Streams im Dokument gespeichert. Der Vorteil des PDF-Formats ist, dass es fast jede Form von elektronischem Dokument speichern kann und es den Nutzern ermöglicht, sie einfach anzeigen zu lassen und zu transportieren. Außerdem ist es unabhängig von der genutzten Umgebung möglich, die Dokumente zu verarbeiten (International Organization for Standardization, 2017). Der Inhalt von bildbasierten PDF-Dateien kann allerdings nicht durch Maschinen erfasst werden. Darum ist es sinnvoll im nächsten Schritt eine Zeichenerkennung mittels OCR durchzuführen. In diesem Schritt werden die Dokumente in JSON- oder das Markdown-Format transformiert. Beides sind weit verbreitete Formate, die sowohl für den Menschen als auch den Computer gut zu erfassen sind (Auer et al., 2024, S. 2–4). Dank der Transformation in austauschfreundliche Formate wie zum Beispiel JSON oder Markdown wird die Weiterverarbeitung in diversen externen Prozessen möglich und die Einspeisung in unternehmenseigene Anwendungen. Die transformierten Dokumente lassen sich einfach maschinell durchsuchen und erlauben es dadurch darin enthaltenes Wissen effizient zu extrahieren.

OCR ist eine Technologie zur Transformation von gescannten Dokumenten oder handgeschriebenem Text aus PDFs oder Bildern in maschinencodierten Text. Der Prozess der Transformation erfolgt schrittweise. Zunächst wird das vorliegende Dokument nach den Komponenten untersucht aus denen es zusammengesetzt ist und entsprechend ge-

gliedert. Die Elemente der Gliederung werden jeweils als Binary Large Object (binäre Großdatenobjekte in Datenbanken) (BLOB) gespeichert. In diesen BLOBs werden die einzelnen Textzeilen ermittelt und anschließend durch Untersuchung der Abstände zwischen den Elementen einer Zeile die Wörter. Im nächsten Schritt wird versucht die Wörter zu erkennen. Unterschieden wird in Wörter die gut erkannt wurden und in Wörter die nicht gut erkannt wurden. Die gesammelten Informationen werden genutzt, um das System zu trainieren und die Erkennung von Wörtern über das Dokument hinweg stetig zu verbessern. Der Schritt wird ein zweites Mal durchgeführt, um mit Hilfe des Trainings Wörter die zuvor nicht gut erkannt wurden möglicherweise besser erkennen zu können (Smith, 2007).

2.2.2 Retrieval Augmented Generation (RAG)

Besonders interessant sind die Dokumente als Wissensbasis in der RAG. Systeme generativer Künstlicher Intelligenz im Bereich der Textverarbeitung die vortrainiert sind können dadurch angepasst werden. Bei der Beantwortung von Fragestellungen greift das System auf seine antrainierte Wissensbasis zurück nutzt aber auch die durch die Anpassung vorhanden unternehmenseigenen Informationen. Dies verbessert die Leistung von generativen KI-Anwendungen erheblich und sorgt für deutliche bessere Ergebnisse (Auer et al., 2024, S. 5). Mit RAG bezeichnet man die Integration von neuen Informationen in den Prozess der Textgenerierung von generativen KI-Anwendungen zusätzlich zu der antrainierten Wissensbasis. Bevor die Anwendung eine Antwort für den Nutzer formuliert sucht sie nach Informationen, die der Beantwortung der Frage dienen. Im Gegensatz zu klassischen LLMs verlassen sich RAG-Anwendungen nicht ausschließlich auf statische Informationen, sondern durchsuchen aktiv angebundene Quellen.

2.2.3 Chunking

Das Chunking ist der erste Schritt, um Textdokumente nutzbar zu machen für RAG-Anwendungen. Die Informationen aus den Dokumenten müssen in einer Vektordatenbank abgelegt werden, um durchsucht werden zu können von der Anwendung. Chunking bedeutet den rohen Text in kleinere semantisch sinnvolle Bausteine zu unterteilen. Dabei ist die Art und Weise und die Größe der produzierten Bausteine entscheidend für das spätere Ergebnis der Informationsgewinnung. Es gibt unterschiedliche Strategien zur Unterteilung von Texten. Diese sind abhängig von der Art des Dokuments, der damit verbundenen Aufgabe und der Struktur des LLMs (MongoDB, Inc., 2025).

Natural language-aware chunking

Diese Methode nutzt die Analyse der natürlichen Sprache, als Natural Language Processing (Verarbeitung natürlicher Sprache) (NLP) bezeichnet, um Dokumente in Bausteine zu teilen. Dabei sollen die Bausteine aus sinnvollen Sätzen bestehen und keine unnatürlichen Brüche besitzen an der Grenze von einem Baustein zum nächsten. Dadurch

ist gewährleistet, dass der transformierte Text kohärent und lesbar bleibt (MongoDB, Inc., 2025).

Paragraph-based chunking

Dieser Ansatz sorgt dafür, dass Texte in logisch abgegrenzte Abschnitte unterteilt wird. Zusammengehörige Ideen werden dabei gemeinsam verarbeitet und der Kontext erhalten. Dieser Ansatz ist sinnvoll in Aufgaben, in welchen der breite Kontext von Informationen notwendig ist (MongoDB, Inc., 2025).

Sliding window chunking

Bei diesem Ansatz überlappen sich die Textbausteine an den Grenzen. Dadurch bleibt der Kontext zwischen Bausteinen erhalten. Nützlich ist dies in Systemen die Kontinuität erfordern, beispielsweise bei der Textgenerierung oder in Dialogsystemen (MongoDB, Inc., 2025).

2.2.4 Tokenization

Mit Tokenization bezeichnet man die Unterteilung des zu Grunde liegenden Textes in kleiner Einheiten. Die dabei entstehenden Tokens sind die kleinsten Einheiten, die ein LLM benutzt. Tokens können Wörter, Wortbruchstücke oder einzelne Buchstaben. Tokens erhalten dann eine numerische Entsprechung die durch das LLM interpretiert werden können.

Der Zusammenhang zwischen Chunking und Tokenization ist für RAG-Anwendungen relevant. LLMs besitzen ein Maximum an Token, dass sie pro Anfrage verarbeiten können. Es gibt ein Limit sowohl für die aufgenommene Menge an Text für eine Anfrage als auch eins für die Abgabe. Dieses Limit wird als "context window length", "context window", "context lengthöder maximum sequence length bezeichnet (IBM, 2025a). Die Aufteilung des Textes in Bausteine, das sogenannte Chunking richtet sich nach der Größe des "Context Window".

2.2.5 Embedding

Das Embedding (zu deutsch: Einbettung) ist ein Weg, um Objekte wie zum Beispiel Text als Punkte in einem fortlaufenden Vektorraum darzustellen. Die Lage eines solchen Punktes gibt ihm eine Bedeutung. Durch die Lage in diesem Vektorraum wird die Bedeutung eines Datenpunktes im Zusammenhang mit anderen Datenpunkten für die Maschine erfassbar. Es gibt zahlreiche Algorithmen wie beispielsweise neuronale Netzwerke mit denen ein Model Embeddings erlernen kann. Durch die Embeddings kann das Model Muster und Beziehungen in Daten erkennen. Objekte die eingebettet werden können sind unter anderem Wörter, Text, Bilder, Audio oder Grafiken. Die Objekte werden umgewandelt in eine numerische Darstellung. Das Ergebnis ist ein Vektor, also

eine Reihe von Zahlen. Vergleicht man zwei unterschiedliche Embeddings bedeutet ein kleiner Abstand zwischen den Vektoren einen großen Zusammenhang zwischen beiden (IBM, 2025b).

2.2.6 Layoutanalyse und semantische Strukturierung von Dokumenten

Für die Verarbeitung von Dokumenten ist nicht nur die reine Erkennung der Zeichen relevant sondern auch die Position an der sich diese befinden. Ein Dokument besitzt eine komplexe 2-Dimensionale Struktur. Die Anordnung von Informationen, das Layout und die Größe von Blöcken kann entscheidend sein für das Verständnis eines Dokuments und dem Leser Hinweise dafür geben wie er die Informationen zu verarbeiten hat. Im Gegensatz zur OCR, die in Bilddateien Text erkennen und diesen extrahieren kann, wird durch die Layoutanalyse des Dokuments die Textsegmente und ihre Anordnung zueinander erfasst (Katti et al., 2018).

Das Verfahren der Document Layout Analysis unterteilt das Dokument in inhaltlich zusammengehörige Abschnitte. Hierbei werden Grafiken, Bilder, Blocktext und Formulare als eigene Blöcke dargestellt. Erste Verfahren haben ausschließlich zweidimensionale Eigenschaften wie Linien, Projektionen oder Abstände berücksichtigt. Neu entwickelte Ansätze dagegen setzen Machine-Learning-Algorithmen ein, die sowohl räumliche als auch inhaltliche Bedeutung beachten (Xu et al., 2020).

Eine große Rolle spielt die Erkennung von Key-Value-Paaren. Diese Art von Information taucht insbesondere im Kontext der DRV häufig auf. Zentrale Informationen wie beispielsweise Vorname, Nachname, Versicherungsnummer, Wohnort oder Krankenversicherung werden von jedem Versicherten gespeichert. Um die Zusammengehörigkeit von Schlüssel und Information zu erkennen ist es notwendig die Art der Anordnung, die Orientierung des Textes und die Abstände zwischen den Werten im Dokument zu ermitteln. Aus diesem Grund werden diese Daten in einem gemeinsamen Block abgespeichert (Katti et al., 2018). Das LayoutLM ist ein Beispiel für ein Modell, das Layoutinformationen zusammen mit Text erfasst (Xu et al., 2020).

Neueste KI-Modelle der Dokumentenverarbeitung kombinieren Modelle für visuelle Erfassung von Dokumenten mit Modellen für Sprachanalyse. In LayoutLM werden die herkömmlichen Sprachmodelle so erweitert, dass die zweidimensionale Position von Tokens in der Darstellung der Information mitberücksichtigt wird.

Die sprachlich-visuelle Darstellung eines Dokuments bildet eine wichtige Grundlage für RAG-Anwendungen. Durch die Unterteilung in inhaltlich zusammenhängende Einheiten kann das Ergebnis des Chunkings verbessert werden und dadurch die Qualität der Informationsgewinnung in Vektordatenbanken. Im Prozess der digitalen Antragsaufnahme in der DRV kann die Verbindung von OCR, Layoutanalyse und mulitmodalen Modellen eine zuverlässige Informationsgewinnung aus verschiedenartigen Dokumenten ermöglichen.

Durch die Layoutanalyse wird die Qualität der Ergebnisse, die die entwickelte Anwendung liefert, stark verbessert. Die eingegebenen Dokumentenstapel besitzen unterschiedlich Arten von Dokumenten, deren Analyse und Zusammenführung essenziell ist für das Resultat.

2.2.7 Human-in-the-Loop

Die Entwicklung von Sprachmodellen besteht aus dem Training, dem Finetuning und dem Test durch die Entwickler. Ist der Test der Funktionsweise erfolgreich gewesen kann die Anwendung eingesetzt werden, um reale Probleme zu lösen. Zu wenig beachtet wird allerdings die Nutzung von menschlichem Feedback. Zum einen ist es möglich durch Nutzerfeedback schon im Entwicklungsprozess Sprachmodelle an die Bedürfnisse von Nutzern anzupassen. Andererseits können trotz umfangreicher Tests Fehler auftreten, die im Vorfeld nicht vorhersehbar sind (Wang et al., 2021). Mit Human-in-the-Loop (HITL) bezeichnet man die Fähigkeit von KI-Systemen in jedem Schritt dem menschlichen Nutzer eine Möglichkeit zur Intervention zu geben (European Commission, 2019). Nutzer sollen die Möglichkeit erhalten die Kontrolle zu übernehmen, um Fehler der Anwendung korrigieren zu können. Darüber hinaus ist es in vielen Prozessen notwendig eine Entscheidung durch den Menschen zu veranlassen. Gerade in Bereichen in denen kritische Personendaten behandelt werden und Entscheidungen Konsequenzen haben für das Schicksal von Menschen muss eine menschliche Instanz die KI kontrollieren. In der Literatur wird der Begriff HITL noch weiter gefasst und bezeichnet damit die fortwährende Integration von Nutzerfeedback in die Weiterentwicklung von Sprachmodellen. Die Verbindung aus KI und Mensch sorgt für eine höhere Genauigkeit, Stabilität in Anwendungen und schafft Vertrauen für den Nutzer (Wang et al., 2021). Gerade in Verwaltungsprozessen bei der DRV ist dies von besonderem Interesse, da in diesem Bereich keine Fehler passieren dürfen. Mehrere Herausforderungen im Zusammenhang mit der Verarbeitung von natürlicher Sprache profitieren von HITL.

Klassifizierung von Text

Ein bekanntes Problem im Bereich NLP ist die Einordnung von Texten in verschiedene Kategorien. Dafür wird ein Modell zur Klassifizierung von Texten trainiert. Basierend auf der aktuellen Ausgabe werden Menschen eingesetzt, um diese zu korrigieren und gegebenenfalls fehlerhafte Einordnungen des Modells zu verbessern. Mit dem vergrößerten Satz an Trainingsdaten wird dann das Modell fortlaufen trainiert, um das Modell sukzessive zu verbessern. Häufig wird eine "Bag-of-wordsStrategie zur Klassifizierung von Dokumenten angewendet. Dabei wird eine Kategorie definiert durch eine beliebige Anzahl an unsortierten Begriffen. Ignoriert werden dabei die semantischen Zusammenhänge und die Reihenfolge in der Wörter im Text auftreten. Eine Weiterentwicklung dieses Ansatzes nutzt eine Art Lexikon, welche Begriffe anhand ihrer Bedeutung zusammenfasst. Gut definierte Lexika liefern weitaus bessere Ergebnisse als reine "Bags-of-words"(Wang et al., 2021).

Gliederung von Text und Named-Entity-Recognition (Eigennamenerkennung) (NER)

Große Verbesserungen durch HITL lassen sich bei der syntaktischen Einordnung und Identifizierung von alleinstehenden Begriffen beobachten. Bei der Verarbeitung von natürlicher Sprache ist es häufig sinnvoll die grammatikalische Struktur von Texten zu analysieren, um Sätze sinnvoll in Abschnitte zu gliedern und eine Beziehung zwischen Wörtern herzustellen. Das hilft bei der Übersetzung von Texten, bei Chatbot-Anwendungen die auf diesen Texten basieren und der Informationsgewinnung. Ebenso nützlich ist die Identifizierung von speziellen Entitäten in Texten, wie beispielsweise Namen oder Ortsbezeichnungen. In beiden Anwendungsfeldern lassen sich durch die Integration von Nutzerfeedback die zu Grunde liegenden Modelle stark verbessern (Wang et al., 2021).

Bildung von Oberbegriffen

Ähnlich wie sich Texte einer Kategorie zuordnen lassen, kann auch für einen Satz von Texten ein passender Oberbegriff gefunden werden, um große Dokumentenstapel zu analysieren und zu ordnen. Unterschiedliche Ansätze lassen den Nutzer die Gewichtung von Wörtern abändern, um die Einordnung des Textes durch das Modell zu verändern. Durch die Einbeziehung des Nutzerfeedback werden die Modelle so trainiert, dass sie bessere Ergebnisse bei der Gruppierung von Texten liefern. Dafür wurde beispielsweise eine Anwendung entwickelt, mit der Nutzer per up-vote oder per down-vote dem System mitteilen konnten, ob ihnen ein Dokumente gefällt oder nicht. (Wang et al., 2021)

Zusammenfassung von Texten

Sprachmodelle zur Textzusammenfassung können verbessert werden, indem die Ausgaben zweier verschiedener Modelle durch den Nutzer miteinander verglichen werden. Das System erfasst dabei die Präferenzen des Nutzers. Darauf aufbauend wird ein Belohnungsmodell trainiert, das antizipiert, welcher von zwei genierten Texten dem Nutzer eher zusagt. Durch “trial-and-error” entwickelt das Modell eine Strategie mit der es zunehmend häufiger die bevorzugte Ausgabe identifizieren kann (Wang et al., 2021).

2.3 Modernisierung der DSRV

Im Jahr 2023 hat der Bundesrechnungshof davor gewarnt, dass der DRV beziehungsweise der DSRV in Würzburg eine Katastrophe im IT-Bereich bevorsteht. Die Technik, die in der DRV eingesetzt wird, ist veraltet und gefährdet deshalb die Funktionsfähigkeit. Obwohl das Problem schon lange bekannt ist blieb die DRV Bund untätig. Bei der DSRV werden die Daten von insgesamt über 70 Millionen Menschen verwaltet, wovon 50 Millionen Versicherte sind und 20 Millionen Rentner. In keiner anderen Behörde werden mehr Sozialdaten verwaltet als hier. Jedes Jahr werden über 2 Millionen Anträge

auf Leistungen gestellt, die verarbeitet werden müssen und die Tendenz ist steigend (IT-Finanzmagazin, 2023). Die DSRV ist im Jahr 1975 in Würzburg entstanden und dient allen Trägern der DRV als gemeinsame Datenstelle. Durch die gesteigerte Mobilität sind die Versicherungsverläufe differenzierter geworden. Häufig hat die Zuständigkeit des regionalen Trägers gewechselt oder des gesetzlichen Versicherungszweigs. Dadurch waren die entsprechenden Stellen mir unvollständigen Versicherungsverläufen konfrontiert. Aus diesem Grund herrschte untereinander ein reger Datenaustausch. Um diesen besser kontrollieren zu können wurde mit der DSRV eine zentrale Datenstelle gegründet. Sie dient allerdings nicht nur zum nationalen Austausch zwischen den Trägern, sondern auch als Kommunikationspartner für externe Stellen wie beispielsweise dem Zoll, der Justiz oder den Kommunen und als Schnittstelle ins Ausland.
Im Sozialgesetzbuch sind die konkreten Aufgaben der DSRV als dateiführende Stelle genannt. Seit 2005 vergibt Sie zentral für alle Träger die Versicherungsnummern und ist verpflichtet zur Führung einer Stammsatzdatei, einer Basisdatei für Arbeitgeberprüfung, einer Basisdatei für die Prüfung bei den Zahlungspflichtigen und einer Datei mit Sofortmeldungen. Außerdem wird eine Datei zur Prüfung der A1-Bescheinigungen geführt. Neben diesen grundlegenden Aufgaben gibt es eine Vielzahl an weiteren Aufträgen die das Sozialgesetzbuch vorschreibt (Deutsche Rentenversicherung Bund, 2025).

Die Anwendungen, die für den Austausch der Daten entwickelt wurden stammen überwiegend aus der Zeit nach der Gründung der DSRV und laufen auf Großrechnern von IBM. Erst im letzten Jahr ist ein neuer Vertrag geschlossen mit der Firma IBM über den Kauf des neuesten Großrechners IBM z17 und der Wartung bis ins Jahr 2031 (Auftrag-Select, 2026). Diese auch als Mainframes bezeichneten Rechner besitzen ein eigenes Betriebssystem z/OS. Dieses Betriebssystem ist speziell für hochverfügbare, transaktionsintensive und geschäftskritische Anwendungen konzipiert. z/OS stellt eine integrierte Umgebung für Batch-Verarbeitung, Online-Transaktionen sowie Datenbankanwendungen bereit und unterstützt dabei Mechanismen zur Ressourcenisolation, Workload-Steuerung und Fehlertoleranz. Charakteristisch ist die Fähigkeit, große Datenmengen mit hoher Zuverlässigkeit zu verarbeiten und gleichzeitig eine kontinuierliche Verfügbarkeit sicherzustellen. Anwendungen sind häufig in Common Business-Oriented Language (COBOL) implementiert und werden entweder im Batch-Modus oder innerhalb von Transaktionsmonitoren wie Customer Information Control System (CICS) ausgeführt. Die enge Verzahnung von Hardware, Betriebssystem und Middleware ermöglicht eine robuste und skalierbare Plattform, die insbesondere im Finanz- und Verwaltungssektor weit verbreitet ist (IBM, 2015).

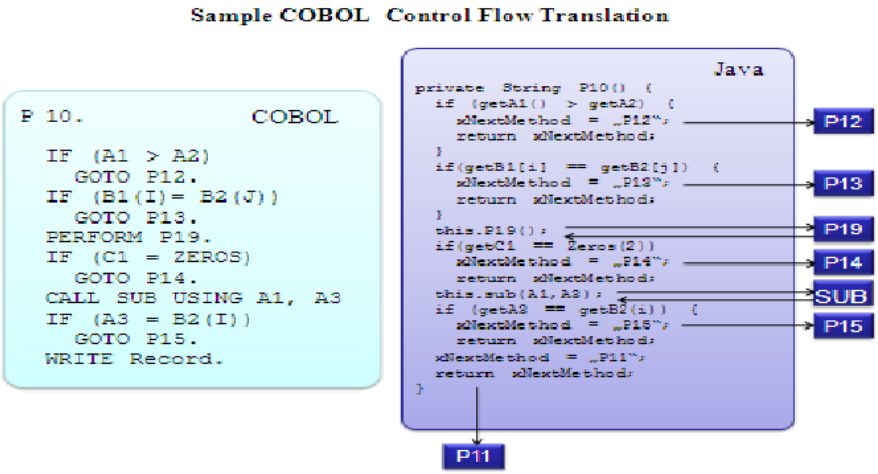

Abbildung 2.5: Transformation von COBOL-Code zu Java-Code (Sneed, 2000)

Mittlerweile entspricht diese Technik allerdings nicht mehr den üblichen Standards. Außerdem passt sie nicht mehr in das IT-Gesamtkonzept in der DRV. Für eine funktionierende Pflege der IT-Infrastruktur ist eine einheitlich Hard- und Software notwendig. Ein Versuch der Modernisierung ist im Jahr 2019 gestartet worden . Dieser wurde allerdings wieder beendet, da man erkannte das eine viel umfangreichere Modernisierung notwendig ist. Inzwischen geht man davon aus, dass es noch mindestens bis ins Jahr 2030 dauert bis die Systeme der DRV umgestellt sind.
Die Notwendigkeit einer Modernisierung ist schon seit vielen Jahren bekannt. Schon im Jahr 2012 ist wurde festgestellt, dass in Zukunft Personal fehlen wird und die drohenden Folgen gravierend sind. Trotz umfangreicher Versuche Personal zu gewinnen, herrscht immer noch Personalmangel. Außerdem sind der DRV zunehmend weitere Aufgaben durch den Gesetzgeber übertragen worden, was die Verfahren deutlich komplexer macht. Die Folge davon ist eine lückenhafte Dokumentation der IT-Systeme, die nun die Modernisierung verlangsamt. An vielen Stellen reicht das interne Wissen nicht mehr aus, um den Betrieb aufrecht zu erhalten. Daher wird auf teure externe Kräfte zurückgegriffen. Der branchenübergreifende Fachkräftemangel und das geringe Interesse am Studium alter Technologie erschwert die Personalgewinnung. Man geht davon aus, dass der fortlaufende Betrieb alter Technik hohe Mehrkosten verursacht. Es hat sich herausgestellt, dass die Modernisierung IT-Infrastruktur beinahe eine komplette Neuentwicklung darstellt. Die Ablösung von Teilen des Bestandssystems durch Neuentwicklungen erfolgt kontinuierlich. Das heißt, dass der Betrieb noch über eine lange Zeit doppelt betrieben werden muss. Eine Alternative zur Neuentwicklung gibt es nicht, da man sich zu lange auf das Know-How der Mitarbeiter verlassen hat und zu wenig dokumentiert wurde. Sollte die Modernisierung fehlschlagen wird es in Zukunft kein Personal zur Verfügung stehen, dass die zentralen Anwendungen betreut. Dadurch droht ein Ausfall des gesamten Systems und gefährdet Einziehung und Auszahlung von Beiträgen beziehungsweise Renten in Milliardenhöhe.

Die Herausforderung besteht darin, dass die Neuentwicklung im Rahmen von DSRV.move im Kontext einer Jahrzehnte gewachsenen, hochkomplexen Großrechner-Landschaft geschieht, die tief in die zentralen Geschäftsprozesse der DRV eingebunden ist. Neben der festgelegten Ablösung des Großrechners im Jahr 2030 ist der Mangel an Fachpersonal im Bereich COBOL und z/OS ein großes Problem. Betont werden muss, dass eine bloße technische Übertragung bestehender COBOL-Anwendungen nicht ausreicht, da dadurch technische Schulden fortgeschrieben oder neue Abhängigkeiten geschaffen werden. Die Migration zu Java und einer verteilten, containerbasierten Architektur ist nicht nur eine technologische Erneuerung, sondern ein notwendiger Schritt zur Sicherung von Wartbarkeit, Innovationsfähigkeit, Personalgewinnung und langfristiger Zukunftsfähigkeit der DRV.

2.3.1 Neuentwicklung der digitalen Antragstellung

Die Neuentwicklung der digitalen Antragstellung im Projekt eAQ:modern erfolgt im Rahmen der Modernisierung der gesamten IT-Landschaft in der DRV.

Das Antragssystem der Deutschen Rentenversicherung (DRV) basiert auf dem im Sozialrecht verankerten Antragsprinzip gemäß § 19 SGB IV. Danach sind Leistungsansprüche grundsätzlich aktiv durch die Versicherten geltend zu machen, da Sozialleistungen nur auf Antrag gewährt werden. Dieses Prinzip prägt die organisatorische und technische Ausgestaltung der Antragsverfahren maßgeblich.

Zur Bewältigung von jährlich rund zwei Millionen Leistungsanträgen stellt die DRV das digitale Erfassungsprogramm eAntrag/Expertenversion bereit. Die Anwendung dient der strukturierten Aufnahme von Anträgen in den Bereichen Rente, Versicherung, Rehabilitation sowie im Kontext der Beitragsangelegenheiten. Neben der Datenerfassung umfasst das System Plausibilitätsprüfungen, automatische Vorschlagsmechanismen sowie Unterantragsfunktionen, um eine konsistente und möglichst vollständige Antragstellung sicherzustellen (Hohmann, 2025).

Die hohe Anzahl an Anträgen ergibt sich einerseits aus dem breiten Leistungsspektrum der gesetzlichen Rentenversicherung. Neben Altersrenten umfasst dieses unter anderem Erwerbsminderungs- und Hinterbliebenenrenten, Leistungen zur medizinischen Rehabilitation und zur Teilhabe sowie internationale Rentenangelegenheiten und Versorgungsausgleiche. Andererseits sind die Anspruchsvoraussetzungen komplex und von unterschiedlichen Faktoren abhängig, etwa von gesetzlichen Rahmenbedingungen, Alter, Versicherungszeiten oder gesundheitlichen Voraussetzungen. Diese Differenzierung erfordert eine Vielzahl spezialisierter Formulare und Prüfmechanismen (Hohmann, 2025).

Insgesamt existieren im Kontext der DRV über 70 unterschiedliche Antragsarten, die sich in mehrere Kategorien einteilen lassen: Rentenanträge, Anträge auf Rehabilitations- und Teilhabeleistungen, Verfahren zu Versicherungs- und Beitragsangelegenheiten – beispielsweise zur Kontenklärung von Versicherungszeiten – sowie sonstige Anträge auf Zusatzleistungen wie Zuschüsse zur Krankenversicherung oder Regelungen im Rahmen eines Versorgungsausgleichs (Hohmann, 2025).

Ziel des Projekts eAQ:modern ist es die herkömmliche Anwendung eAntrag/Expertenversion durch die neue Anwendung abzulösen.

Kommunikationsstrecke

Die Kommunikationsstrecke des eAntrag-Verfahrens beschreibt den vollständigen Weg eines Leistungsantrags von der initialen Antragstellung durch den Versicherten bis zur abschließenden Rückmeldung durch den zuständigen Rentenversicherungsträger. Das in Abbildung 2.6 dargestellte Swim-Lane-Diagramm zeigt die Verarbeitung eines Antrags durch die verantwortlichen Stellen beim Träger beziehungsweise der DSRV innerhalb der Systemlandschaft der DRV.

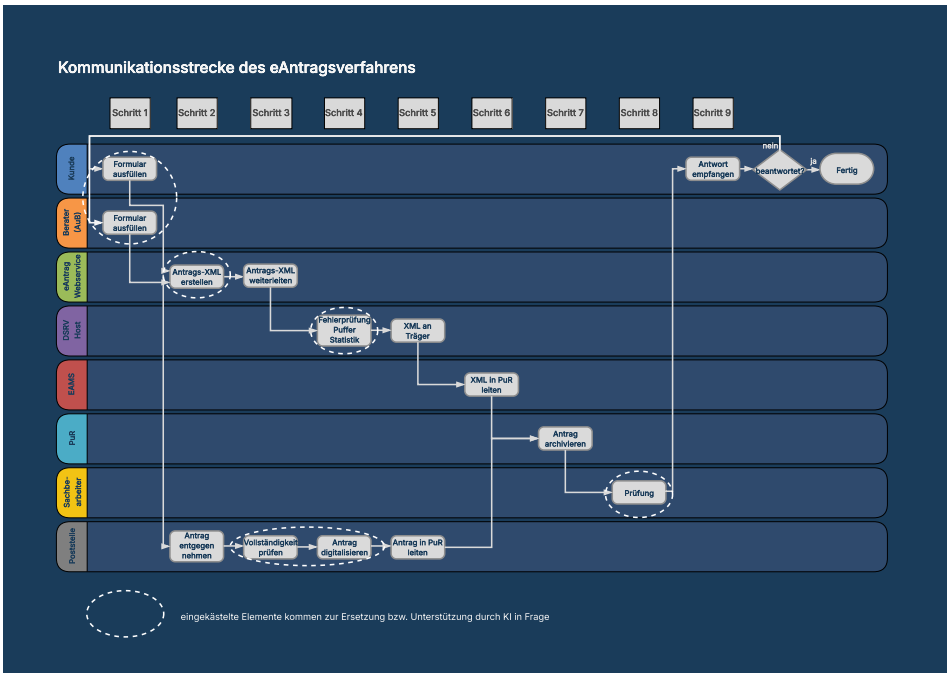

Abbildung 2.6: Kommunikationsstrecke des eAntrag-Verfahrens (Hohmann, 2025)

1. **Antragstellung (Schritt 1)**
   Der Prozess beginnt mit der aktiven Antragstellung durch den Versicherten, was dem sozialrechtlichen Antragsprinzip gemäß § 19 Sozialgesetzbuch (SGB) IV entspricht. Die Antragstellung kann entweder papierbasiert oder elektronisch erfolgen. Elektronische Anträge werden über die eAntrag-Webversion eingereicht, die im Rahmen des Projekts eAQ:modern neu entwickelt wird und perspektivisch sowohl die bisherige Webversion als auch die Expertenversion ablöst (Hohmann, 2025).

   Während der Antragstellung werden personenbezogene Daten, Informationen zum Versicherungsverlauf sowie gegebenenfalls Anlagen wie ärztliche Atteste oder Bescheinigungen erfasst. In der elektronischen Variante erfolgen bereits Plausibilitätsprüfungen der Formularfelder. Eine inhaltliche Prüfung oder Klassifikation

der eingereichten Anlagen findet jedoch derzeit noch nicht automatisiert statt (Hohmann, 2025)

2. **Erstellung und Übermittlung des Antragsdatensatzes (Schritte 2 und 3)**
   Nach Abschluss der Eingabe wird aus den Formulardaten ein strukturierter Antragsdatensatz im Extensible Markup Language (XML)-Format generiert. Dieser besteht aus einem fachlichen Teil (Formulardaten und Anlagen) sowie einem technischen Teil mit Steuerinformationen zur internen Weiterleitung, insbesondere der Versicherungsnummer (VSNR) (Hohmann, 2025).

   Die Übermittlung erfolgt über den eAntrag-Webservice an den zentralen Single Point of Contact (SPoC) der DSRV. Der Versicherte erhält eine Sendebestätigung. Der Datensatz wird anschließend an die zuständigen Fachdienste weitergeleitet, unter anderem zur statistischen Verarbeitung (Hohmann, 2025).

3. **Verteilung an den zuständigen Rentenversicherungsträger (Schritte 4 und 5)** Anhand der Versicherungsnummer wird der zuständige Rentenversicherungsträger ermittelt. Der strukturierte XML-Datensatz wird an den dortigen SPoC übertragen und in das Systemumfeld des jeweiligen Trägers integriert. Papieranträge werden zuvor eingescannt und als PDF-Dokumente gespeichert (Hohmann, 2025).

   Sowohl elektronische als auch papierbasierte Anträge werden anschließend im Post- und Recherche-Client (rvPuR) bereitgestellt. Die Anwendung „eAntrag und mittlere Serverebene (EAMS)-Eingang“ bereitet die eingehenden Daten für die weitere Bearbeitung auf (Hohmann, 2025).

4. **Speicherung im Kernsystem (Schritte 5 und 6)** Der Antragsdatensatz wird im Kernsystem des zuständigen Trägers gespeichert und archiviert. Während elektronische Anträge als strukturierte XML-Daten vorliegen, bestehen Papieranträge lediglich aus gescannten PDF-Dokumenten ohne strukturierte Inhaltsrepräsentation (Hohmann, 2025).

   Diese unterschiedliche Datenstruktur hat erhebliche Auswirkungen auf die Weiterverarbeitung: Strukturierte Daten ermöglichen eine maschinengestützte Auswertung, während PDF-Dokumente eine manuelle Sichtung erfordern (Hohmann, 2025).

5. **Sachbearbeitung (Schritt 8)** Die fachliche Prüfung erfolgt durch Sachbearbeiter über die Anwendung „rvDialog“, welche auf den rvPuR zugreift. Offene Anträge werden dort angezeigt und im Rahmen einer individuellen Fallprüfung bearbeitet. Dabei werden sowohl die Angaben im Antragsdatensatz als auch die eingereichten Anlagen inhaltlich geprüft (Hohmann, 2025).

   Insbesondere die manuelle Sichtung und Indizierung von Anlagen stellt einen zeitintensiven Arbeitsschritt dar. Fehlende oder unplausible Angaben führen zu Rückfragen an den Versicherten (Hohmann, 2025).

6. **Rückmeldung an den Versicherten (Schritt 9)** Nach Abschluss der Prüfung erhält der Versicherte eine schriftliche Rückmeldung per Post. Diese kann eine Bewilligung, eine Ablehnung oder eine Aufforderung zur Nachreichung weiterer Unterlagen enthalten. Ist der Sachverhalt abschließend geklärt, wird der Vorgang im System abgeschlossen (Hohmann, 2025).

Die Betrachtung der gesamten Kommunikationsstrecke ist insbesondere deshalb relevant, weil das Projekt eAQ:modern zwar primär die Neuentwicklung der Webanwendung umfasst, jedoch in eine komplexe Systemlandschaft eingebettet ist. Entscheidungen zur Implementierung von KI-gestützten Funktionen, etwa zur Indizierung von Anlagen oder zur automatischen Datenextraktion, wirken sich nicht nur auf die Antragstellung, sondern auf mehrere nachgelagerte Prozessschritte aus (Hohmann, 2025).

Die Analyse der Kommunikationsstrecke zeigt, dass insbesondere die frühen Prozessschritte (Antragserfassung, Anlagenverarbeitung, Datenstrukturierung) entscheidend für die Effizienz der nachgelagerten Sachbearbeitung sind. Hier besteht das größte Potenzial zur Reduktion manueller Tätigkeiten und zur Entlastung der Fachkräfte.

3 Projektdurchführung

3.1 Projektbeschreibung

Die in dieser Bachelorarbeit vorgestellte Anwendung ist, als zusätzliches Feature, Teil der neu zu entwickelnden digitalen Antragstellung in der DRV. In den ersten Schritten der eAntrag-Kommunikationsstrecke besteht der größte Nutzen für den Einsatz von KI zur Automatisierung, da im Bereich der Antragsaufnahme, Anlagenverarbeitung und Datenstrukturierung viele Daten manuell zu verarbeiten sind.

Entwickelt wird eine responsive Web-Anwendung in Form einer SPA zur intelligenten Dokumentenverarbeitung. Verarbeitet werden Dokumentenstapel im PDF-Format, die aus einer beliebigen Anzahl von Einzeldokumenten bestehen. Diese zeichnen sich dadurch aus, dass sie sowohl gedruckten als auch handschriftlichen Text enthalten. Informationen sind in Form von Tabellen, Textblöcken, Bildern und Grafiken enthalten. Für das Verständnis der verschiedenartig codierten Informationen ist neben einer inhaltlichen auch eine strukturelle Interpretation notwendig. Eine mehrschichtige Pipeline zerteilt die Dokumentenstapel in alleinstehende Dokumente, transformiert diese in maschinenlesbare Formate und extrahiert Entitäten. Die eingegebenen Daten werden persistiert und innerhalb der Anwendung verwaltet, sodass sie dem Nutzer zur späteren Bearbeitung zur Verfügung stehen. Die Benutzeroberfläche dient der Betrachtung und Auswahl von Dokumentenstapel. Eine Wizard-basierte Nutzerführung führt schrittweise durch den Transformationsprozess. Dabei wird in jedem Schritt nach dem HITL-Prinzip die Kontrolle an den Nutzer übergeben und Feedback eingeholt. Nach der Verarbeitung liegen die extrahierten Informationen in strukturierter Form vor und lassen sich automatisiert verarbeiten. Die Anwender werden unterstützt bei ihrer zeitaufwendigen und fehleranfälligen Verarbeitung von Dokumenten.

Die Architektur folgt dem Client-Server-Prinzip, bei dem Präsentationslogik und Geschäftslogik klar separiert sind. In zwei eigenständigen Projekten wird ein Client und ein Server entwickelt. Der Client implementiert die Benutzeroberfläche, leitet Anfragen an den Server weiter und stellt die empfangen Informationen in der Oberfläche dar. Der Server bietet eine Representational State Transfer (REST)-API an über welche Daten per HTTP empfangen und gesendet werden. Die empfangenen Daten werden durch den Server verarbeitet, eine Datenbank wird für die Datenhaltung angebunden und per REST-API werden externe Dienste für die intelligente Dokumentenverarbeitung genutzt.

3.2 Technische Rahmenbedingungen

Für die Entwicklung wird ein stationärer und ein mobiler Rechner mit Windows 11 als Betriebssystem verwendet. Als integrierte Entwicklungsumgebung wird IntelliJ IDEA eingesetzt. Diese dient der Verwaltung der Projektstruktur, der Quellcodedateien sowie der Implementierung von Frontend und Backend.

Mit Hilfe der Versionsverwaltungssoftware Git werden die unterschiedlichen Entwicklungsstände fortlaufend synchronisiert. In Github werden die beiden Projekte für Frontend beziehungsweise Backend angelegt, gespeichert und verwaltet. Für die Verwaltung der lokalen Git-Repositories wird der für Windows entwickelte Git-Client *Fork* verwendet.

Das Framework Angular dient der Entwicklung des Frontends, das als SPA umgesetzt wird. Die Abhängigkeitsverwaltung erfolgt über npm. Für die Entwicklung des Backends wird das Spring-Boot-Framework verwendet. Die Frameworks bieten in vorgefertigten Bibliotheken wiederkehrende Standardfunktionen an. Dadurch können sich Entwickler auf die Entwicklung der Geschäftslogik konzentrieren und der Entwicklungsprozess wird effizienter.Das Backend wird mittels Maven als Build- und Dependency-Management-Werkzeug verwaltet.

Die Datenhaltung im Backend erfolgt in einer containerisierten PostgreSQL-Datenbank. Durch die Ausführung in einem Docker-Container findet eine Kapselung statt und die Datenbank ist leichter zu reproduzieren. Die Persistenzschicht wird über Spring Data JPA realisiert, wodurch Datenbankzugriffe abstrahiert werden. Die Orchestrierung der Container erfolgt über docker-compose.

In einem weiteren Docker-Container findet die Transformation der PDF-Dateien in JSON oder Markdown statt. *Docling Serve* erlaubt es einen Docling-Server in einem Container zu deployen (Contributors, 2025). Dadurch können die Funktionen zur Verarbeitung von Dokumenten per REST-API angesprochen werden. Die Container laufen in einem mit Windows Subsystem für Linux 2 (WSL2) virtualisierten Linux-Betriebssystem lokal auf den Windows-Rechnern.

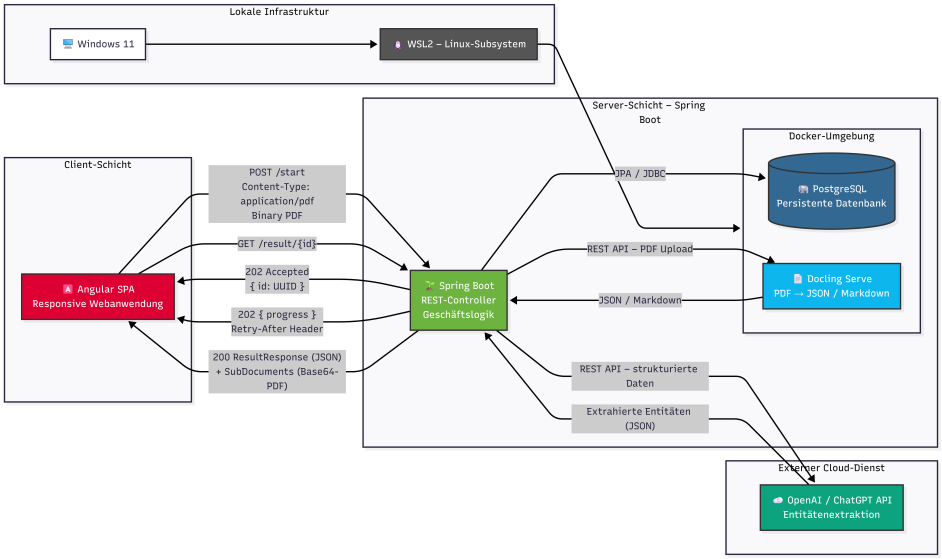

Abbildung 3.1: Systemarchitektur der KI-gestützten Dokumentenverarbeitung

Eine detaillierte Beschreibung der Architektur sowie der einzelnen Komponenten erfolgt in den nachfolgenden Kapiteln.

3.3 Anwendung - Frontend mit Angular

Angular ist ein von Google entwickeltes Web Framework, das Entwicklern hilft schnell zuverlässige Anwendungen zu bauen. Einige essentielle Konzepte werden darin implementiert.

1. **Komponenten**
   Ein Kernelement von Angular sind Komponenten. Eine Webanwendung besteht aus mehreren Komponenten, die dabei helfen das Projekt zu strukturieren und Code in spezielle Funktionseinheiten zu gliedern die sich beliebig oft wiederverwenden lassen. Eine Komponente besteht aus einem Decorator der verschieden Konfigurationen für Angular vornimmt , einem HTML-Template welches die Inhalte des DOM enthält, einem CSS-Selektor wie die Komponente im HTML benutzt wird und einer TypeScript-Klasse in der Funktionalitäten wie die Verarbeitung oder die Anfragen an einen Server implementiert sind.

2. **Services**
   Um Kapselung und Wiederverwendbarkeit zu verstärken wird unterschieden in Komponenten und Services. Die Komponente soll ausschließlich die Benutzeroberfläche kontrollieren. Dazu enthält sie Properties und Methoden um Daten an das Template zu binden. Alle weiteren Funktionalitäten werden an Services delegiert. Services fragen Daten vom Server an, validieren Nutzereingaben oder führen

Logs auf der Konsole durch. Mit Hilfe der Dependency Injection werden die Services den Komponenten verfügbar gemacht.

3. **Dependency Injection**
   Dependency Injection ist ein Kernkonzept von Angular, das die Kapselung von Funktionalitäten in Services ermöglicht. Eine Klasse legt die Instanzen von benötigten Services nicht mehr selbst an. Dies geschieht nun durch einen Dependency Container genannt Injector. Angular implementiert ein zentrales Dependency-Injection-System, das Dependencies zentral verwaltet und bei Bedarf in Komponenten injiziert.

4. **Templates und Data-Binding**
   Templates definieren die DOM-Struktur. Durch sie reagiert die Benutzeroberfläche auf Änderung der Daten. Templates basieren auf der HTML-Syntax besitzen aber zusätzliche Features wie beispielsweise Template-Funktionen, Data-Binding, Event-Listening und Variablen. Das dynamische Verhalten der Benutzeroberfläche wird durch Data-Binding zwischen Template und Daten erreicht.

5. **Directives**
   Directives sind Erweiterungen im HTML, die zur dynamischen Manipulation der DOM-Struktur oder zur Modifikation von Elementattributen eingesetzt werden. Sie ermöglichen eine zustandsabhängige Darstellung und Interaktion innerhalb der komponentenbasierten Benutzeroberfläche.

6. **Routing**
   In einer SPA ändert sich die Anzeige der Benutzeroberfläche durch einblenden und verstecken von Inhalten, statt neue Seiten vom Server anzufragen. Um die Navigation zwischen Ansichten zu wechseln wird ein Router verwendet. Dieser interpretiert die Browser-URL und ändert die View entsprechend.

7. **HTTP-Client**
   Der Angular HttpClient stellt eine Schnittstelle zur Durchführung von HTTP-Anfragen bereit. Er ermöglicht die asynchrone Kommunikation mit REST-basierten Backend-Diensten. Der HttpClient liefert standardmäßig ein Observable. Wenn ein Beobachter ("Subscriber") sich auf ein Observable registriert hat wird er bei neuen Werten benachrichtigt. JSON-basierte Antworten werden automatisch deserialisiert und typisiert verarbeitet.

3.3.1 Aufbau der SPA

Die SPA besteht aus den drei Seiten Übersicht, Dokumentenstapel und Geschichte der DRV. Eingefasst werden die Seiteninhalte durch einen Header-Bereich am oberen und einen Footer-Bereich am unteren Rand. Der Header-Bereich enthält ein Menü über das zwischen den einzelnen Seiten navigiert werden kann. Am unteren Rand wird die Oberfläche durch einen Footer-Bereich abgeschlossen.

Die Seite Dokumentenstapel, dient als Einstieg in den Prozess der Dokumentenverar-

beitung. Zunächst wird der 3-stufige Prozess für den Nutzer erklärt. Dieser besteht aus dem Hochladen eines Dokumentenstapels, der Prüfung des verarbeiteten Dokumentenstapels und abschließend eine Zusammenfassung der Ergebnisse. Durch Beendigung des Vorgangs wird der hochgeladene Dokumentenstapel unter seiner Vorgangsnummer und den einzelnen Dokumenten gespeichert.

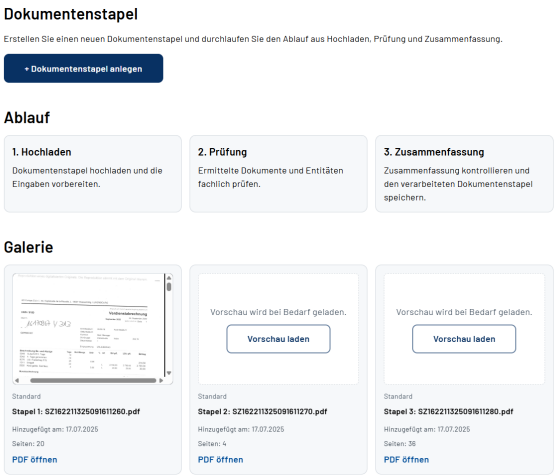

Abbildung 3.2: Einstieg in den Prozess der Dokumentenverarbeitung

Der Einstieg in den Prozess der Dokumentenverarbeitung erfolgt über den Button “Dokumentenstapel anlegen”. Abgeschlossene Vorgänge werden inklusive einer Vorschau aufgelistet und können eingesehen und bearbeitet werden (3.2).

Abbildung 3.3: Geführte Navigation durch den Prozess der Dokumentenverarbeitung

Der 3-Stufen Prozess wird, wie in Abbildung 3.3 zu sehen ist, in einer Wizard-basierten Führung des Nutzers durchlaufen. Die Schritte werden nacheinander abgearbeitet. Bevor ein neuer Schritt aktiviert wird, muss der vorherige abgeschlossen sein. Der Nutzer wird durch klare “Weiter”/“Zurück”-Navigation geführt. Dadurch wird die Komplexität für den Nutzer reduziert. Außerdem ist es dadurch möglich jeden Schritt auf Vollständigkeit zu prüfen bevor er abgeschlossen wird.

3.4 Softwarearchitektur

3.5 Designsystem

3.6 Versionierung

3.7 Anwendung - Backend mit Spring-Boot

3.8 Anwendung - Containerisierung

4 Zusammenfassung

5 Ausblick

Abbildungsverzeichnis

2.1 Architektur von Single-Page-Anwendungen (Tech-Stack.com, 2026) . . 8
2.2 Das MVC-Architekturmuster (Scott, 2015, S. 25) 9
2.3 Das MVVM-Architekturmuster Scott, 2015, S. 27 10
2.4 Transformation von physischen Dokumenten (ComputerWeekly.de, 2025) 11
2.5 Transformation von COBOL-Code zu Java-Code (Sneed, 2000) 18
2.6 Kommunikationsstrecke des eAntrag-Verfahrens (Hohmann, 2025) . . . 20

3.1 Systemarchitektur der KI-gestützten Dokumentenverarbeitung 25
3.2 Einstieg in den Prozess der Dokumentenverarbeitung 27
3.3 Geführte Navigation durch den Prozess der Dokumentenverarbeitung . 27

Abkürzungsverzeichnis

API	Application Programming Interface
AuB	Auskunft- und Beratungsstelle
BLOB	Binary Large Object (binäre Großdatenobjekte in Datenbanken)
CICS	Customer Information Control System
COBOL	Common Business-Oriented Language
CSS	Cascading Style Sheets
DOM	Document Object Model
DRV	Deutsche Rentenversicherung
DSRV	Datenstelle der Deutschen Rentenversicherung
EABE	eAntrag Basisdienste
EAMS	eAntrag und mittlere Serverebene
eAQ:modern	Modernisierungsprojekt des digitalen Antragsverfahrens der DRV
HITL	Human-in-the-Loop
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment(integrierte Entwicklungsumgebung)
IDP	Intelligent Document Processing
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
KNN	Künstliche Neuronale Netze
LLM	Large Language Model
MVC	Model-View-Controller
MVP	Model-View-Presenter

MVVM	Model-View-ViewModel
NER	Named-Entity-Recognition (Eigennamenerkennung)
NLP	Natural Language Processing (Verarbeitung natürlicher Sprache)
OCR	Optical Character Recognition
PDF	Portable Document Format
PDFA	ISO-standardized Portable Document Format for Archiving (PDF/A)
RAG	Retrieval Augmented Generation
RCP	Rich-Client-Platform
REST	Representational State Transfer
RV	Rentenversicherung
rvPuR	Post- und Recherche-Client
SGB	Sozialgesetzbuch
SPA	Single-Page Application
SPoC	Single Point of Contact
VSNR	Versicherungsnummer
WSL2	Windows Subsystem für Linux 2
XML	Extensible Markup Language

Literatur

Auer, C., Lysak, M., Nassar, A., Dolfi, M., Livathinos, N., Vagenas, P., Berrospi Ramis, C., Omenetti, M., Lindlbauer, F., Dinkla, K., Mishra, L., Kim, Y., Gupta, S., Teixeira de Lima, R., Weber, V., Morin, L., Meijer, I., Kuropiatnyk, V., & Staar, P. W. J. (2024). *Docling Technical Report* (Techn. Ber.) (arXiv:2408.09869v5). IBM Research, AI4K Group. Rueschlikon, Switzerland. https://arxiv.org/abs/2408.09869

Auftrag-Select. (2026). *Bekanntmachung zur Ausschreibung (ID 7247464)* [Online-Ausschreibungs-Bekanntmachung]. Verfügbar 5. März 2026 unter https://files.auftrag-select.com/file/auftrag-select-public/pdfs/7247464/bekanntmachung.pdf

ComputerWeekly.de. (2025, 12. April). *Was ist intelligente Dokumentenverarbeitung (Intelligent Document Processing, IDP)?* Verfügbar 27. Februar 2026 unter https://www.computerweekly.com/de/definition/Intelligente-Dokumentenverarbeitung

Contributors, D. (2025, Januar). *Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion* (Techn. Ber.). https://doi.org/10.48550/arXiv.2501.17887

Deutsche Rentenversicherung. (2025). *Rentenversicherung in Zahlen 2025: Statistik der gesetzlichen Rentenversicherung*. Verfügbar 23. Februar 2026 unter https://www.deutsche-rentenversicherung.de/SharedDocs/Downloads/DE/Statistiken-und-Berichte/statistikpublikationen/rv_in_zahlen.pdf

Deutsche Rentenversicherung Bund. (2025). *Studientext Nr. 31: Datenverarbeitung in der Rentenversicherung*.

European Commission. (2019). *Ethics Guidelines for Trustworthy AI* (Techn. Ber.). High-Level Expert Group on AI.

Hohmann, D. (2025, 1. September). *KI-gestützte Antragserstellung – Assistent für den digitalen Antragsprozess der DRV: Machbarkeitsstudie im Rahmen des Projekts eAQ:modern* [Praxisbericht im Bachelor-Studiengang Angewandte Informatik]. Hochschule Kaiserslautern.

IBM. (2015). *Introduction to the New Mainframe: z/OS Basics*.

IBM. (2025a). *Tokens in watsonx.ai* [Accessed 2026-03-01]. IBM Corporation. Verfügbar 1. März 2026 unter https://www.ibm.com/docs/en/watsonx/saas?topic=solutions-tokens

IBM. (2025b). *What Are Embeddings?* [Accessed 2026-03-01]. IBM. Verfügbar 1. März 2026 unter https://www.ibm.com/think/topics/embedding

International Organization for Standardization. (2017). *ISO 32000-2:2017. Document management — Portable document format — Part 2: PDF 2.0*.

IT-Finanzmagazin. (2023). *Modernisierungsprojekt gescheitert – Droht der Deutschen Rentenversicherung ein IT-Kollaps?* [Online-Artikel]. Verfügbar 5. März 2026

unter https://www.it-finanzmagazin.de/modernisierungsprojekt-gescheitert-droht-der-deutschen-rentenversicherung-ein-it-kollaps-165581/

Katti, A. R., Reisswig, C., Guder, C., Brarda, S., Bickel, S., Höhne, J., & Faddoul, J. (2018). Chargrid: Towards Understanding 2D Documents. *Proceedings of EMNLP*.

Lummer, K., Loichinger, E., & Pötzsch, O. (2025, 11. Dezember). *Bevölkerung im Wandel: Ergebnisse der 16. koordinierten Bevölkerungsvorausberechnung: Statement zur Pressekonferenz am 11. Dezember 2025*. Statistisches Bundesamt (Destatis). Verfügbar 23. Februar 2026 unter https://www.destatis.de/DE/Presse/Pressekonferenzen/2025/bevvorausberechnung/statement-bevvorausberechnung.pdf

MongoDB, Inc. (2025). *Chunking Explained* [Zugriff am 01.03.2026]. MongoDB, Inc. Verfügbar 1. März 2026 unter https://www.mongodb.com/resources/basics/chunking-explained

Scott, E. A. (2015). *SPA Design and Architecture: Understanding Single Page Web Applications*. Manning Publications Co. Verfügbar 25. Februar 2026 unter https://books.google.com/books?id=Ud-0jwEACAAJ

Smith, R. (2007). An Overview of the Tesseract OCR Engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 629–633. https://doi.org/10.1109/ICDAR.2007.4376991

Sneed, H. M. (2000). Migrating from COBOL to Java. *IEEE Software*, *17*(2), 30–37. https://doi.org/10.1109/52.841663

Tech-Stack.com. (2026). *Modern Application Development* [Accessed: 2026-02-25]. Verfügbar 25. Februar 2026 unter https://tech-stack.com/blog/modern-application-development/

Wang, Z. J., Choi, D., Xu, S., & Yang, D. (2021). Putting Humans in the Natural Language Processing Loop: A Survey. *arXiv preprint arXiv:2103.04044*. https://doi.org/10.48550/arXiv.2103.04044

Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of Text and Layout for Document Image Understanding. *Proceedings of KDD*.